Unit-1

Machine learning Basics

- learning Algorithms
- capacity
- overfitting and underfitting
- Hyperparameters and validation sets.
- estimators
- Bias and variance
- Maximum Likelihood estimation
- Bayesian statistics
- supervised learning Algo's
- Unsupervised learning Algo's
- stochastic Gradient Deseent
- Building a Machine learing Algorithm
- challenges Motivating Deep learning.

Deep Feedforward Networks

Learning XOR

- Gradient-Based learning.
- Hidden Units.
- Architecture Design
- Backpropagation
- Other Differentiation Algorithms.

Machine learning (ML) is method where a system learns from data instead of being explicitly programmed.
e.g:- Email spam filter learns from past emails → identifies new spam.

Learning Algorithms:-

A learning algorithm is a set of rules or mathmetical steps used to train a machine learning model. It improves its performance ~~on~~ ~~of~~ ~~task~~ ~~by~~ ~~learning~~ ~~from~~ ~~data~~ by minimizing the error/Loss on training data.

components of a learning Algorithm:-

A learning algorithm generally includes:

a) model :- function that maps inputs to outputs.
e.g:- Linear Regression, Neural Networks.

b) parameters (θ)
These are values that the algorithm updates during training (weights + biases in DL)

c) Loss Function
Measures how wrong the predictions are
common losses : MSE, Cross-entropy.

d) Optimization Method
Used to update and improves parameters.
e.g:- Gradient Descent, SGD, Adam.

Process of a learning Algorithm
The learning algorithm follows these steps:-

1. Initialize weights :-
Randomly start with some parameter values.

2. Make predictions (forward pass)
   Input → model → output.
3. Calculate loss
   loss = error between output and actual label
4. Compute Gradients
   find how much each parameter contributed to the error (∂loss/$\partial\theta$).
   In neural networks, gradients are computed using Backpropagation.
5. Update parameters.
   use an optimizer:
   $\theta \leftarrow \theta - \eta *$(gradient)
   (η = learning rate)
6. Repeat :- continue for many epochs until error becomes minimum.

Types of learning algorithms

a. supervised learning Algorithms
   use labeled data
   eg:- • linear/logistic regression
   • Neural Networks
   • support vector Machine
   • Decision Trees.

b. Unsupervised/
   No labels
   eg:- K-means
   • PCA
   • Autoencoders.

c. Reinforcement learning Algos
   learn by reward and punishment
   eg:- Q-learing & policy Gradients.

Gradient Descent family (Main DL Learning Algos)

- Gradient Descent :- updates parameters using full dataset (slow but accurate).
- stochastic Gradient Descent (SGD):-
  updates using one sample at a time (fast)
- Mini-Batch Gradient Descent
  Uses small batches (most common in deep learning)

popular variants:-
• momentum • RMSprop • Adam • Adagrad
• Adadelta.

A learning algorithm is a procedure that adjusts the parameters of a model to minimize loss and improve prediction accracy, typically using methods like gradient descent and back propagation.

Capacity

It is the ability of a model to fit or represent a wide variety of functions. A low-capacity model can represent only simple patterns, while a high-capacity model can represent very complex patterns. Capacity affects underfitting and overfitting.

High capacity model
• can approximate complex function
• can learn very detailed patterns
• How many parameters
• eg:- Deep neural networks with many layers

Low-capacity model
• can learn only simple patterns
• cannot fit complicated relationships
• Has fewer parameters
• Eg:- Linear regression (straight line).

Why is capacity important?

Because capacity decides how well a model fits the training data.

- low capacity → underfitting
  (model too simple → cannot learn the data.
- high capacity — overfitting
  (model too complex → memorizes data)
- Medium capacity → Best fit
  (model learns correctly & generalize well)

Factor that Increases model capacity

1. Number of parameters increases
2. number of layers increases
3. complexity of architecture increases
4. Use of non-linear activation functions
5. More features or input dimensions.

Underfitting :- It occurs when the model is too simple to learn the underlying patterns in the data, resulting in high error on both training and test sets. It is caused by low capacity or in sufficient training.

underfitting occurs when a model:

- Has low capacity
- Fails to capture important relationships.
- Has high bias (wrong assumptions)

Symptoms :-

- High training error
- High test error
- Train accuracy & test accuracy both low

causes :-

- model is too simple (few layers, few neurons)
- Too much regularization (Heavy L1/L2, high dropout)
- Not trained enough (Too few epochs)
- Poor features.

Fix Underfitting

- use a bigger model
- Add more layers / more neurons
- Train longer
- Reduce regularization
- Use better features / add nonlinearity.

Overfitting

It occurs when the model learns the traning data too closely — including noise — which results in low training error but high test error. It is caused by high capacity or insufficient regularization.

overfitting happens when the model :-

- Has very high capacity
- learns even small errors / noise in training data
- Remembers training data instead of learning general patterns.
- Has high variance.

Symptoms

- Very low training error
- High test error
- Big gap between train & test accuracy.

causes :-

- model too big (too many layers)
- Training too long
- Not enough training data
- No regularization
- Noisy or bad data

fix

- Add regularization (L1 / L2 weigh decay)
- Use dropouts
- Use early stoping
- Reduce model size
- Use data augmentation
- Increase training data.

Hyperparameters and Validation sets

Hyperparameters:-

Deep learning models learn from data, but some setting cannot be learned automatically. These setting must be set by us before training. These are called Hyperparameters.

Hyperparameters are external settings that control how the model learns, but are not learned from the data.

Key DL Hyperparameters:-

- Learning Rate (η) :- How big of a step the optimizer takes towards the minimum loss.
- Hidden layers Numbers:- How deep your network is
- Number of Neurons :- How wide your layers are (model capacity).
- Batch size:- How many samples are processed together before updating the weights.

Finding the best set of hyperparameter is called Hyper Parameter Tuning.

Why needed:-

- speed of learning
- model capacity
- overfitting/underfitting
- Accuracy and performance

Choosing wrong hyperparameters can make a good model perform badly.

Validation sets : The "practice exam".

- Subset of data used to tune hyperparameters
- Helps detects overfitting
- Helps decide when to stop training
- Ensures good generalization.

If hyperparameters are the "recipe", the validation set is how you test if that recipe works well before the final exam.

When training a deep learning model, we split data into:

1. Training set (70%) – used to train the model.
2. Validation set (15%) – used to evaluate model while training
3. Test set (15%) – used at the end to check final performance.

The validation set is not used for training.

How they work Together

The validation set prevents overfitting to the training data.

1. You choose a set of Hyperparameters (e.g learning Rate = 0.01)
2. You Train the model on the training set.
3. After every few epochs, you measure the models error on the Validation set.
4. If the training error continues to go down but the validation error starts to go up, that's the single for overfitting.
5. You the stop training (Early stopping) or go back to step 1 and try different set of Hyper Parameters (e.g.. a smaller learning rate).

The Validation set helps you make decisions about the hyperparameters and when to stop training without contaminating the final Test set.

Estimators

An estimators is a method that calculates the best model parameters using data. It measures error (loss), finds the direction to reduce that error (gradient), and updates the weights to improve learning.

→ An estimator is a method that calculates something important from data.
That's it just like:-
- A thermometer estimates temperature
- A clock estimates time.

- An estimator estimates the best model parameters (weights & bias).

Why do we need?

Because-
- The model doesn't know which weights are correct
- It must calculate the best weights using data.
- The method that does this calculation is the estimator.

What an Estimator calculates:-

- An estimator calculates:-
  1. loss (error)
  2. Gradient (direction to reduce error)
  3. updated weights.

Bias and Variance

Machine learning models make errors when predicting outcomes. These errors mainly come from two sources:

1. Bias
2. Variance

Understanding these helps us choose the right model complexity and improve performance.

Bias:-

It refers to the error that occurs when a model is too simple and cannot capture the underlying pattern of the data.

- High bias → model underfits.
- It makes strong assumptions (e.g. assumes a linear shape for non-linear data.

Bias formula:-

It is defined as the difference between the average predicated value and the true value.

$$Bias(x) = E[\hat{f}(x)] - f(x)$$

where:-
- $\hat{f}(x)$ = model predication
- $f(x)$ = true function
- E = expected value (average over many training sets).

Interpretation:-

- If Bias is high, model is oversimplified
- Traing error is high
- Test error is also high
- model does not learn enough → underfitting.

2. Variance (Error due to sensitivity)

Variance refers to how much the model's predictions change if we train it on different samples of data.

- High variance → model overfits.
- Model becomes very sensitive to small fluctuations in training data

Variance formula:-

Variance is:-

$$\text{Variance}(x) = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$$

meaning:-

- first calculate average prediction
- Then measure how far each model's prediction from that average.
- High variance = predictions fluctuate too much.

Interpretion:-

- High variance → model is too complex
- Training error will be low.
- Test error will be high.
- Model learns noise → overfitting.

Bias-Variance Tradeoff

Bias and variance cannot be minimized at same time

- If you reduce bias – model becomes more complex → variance increases
- If you reduce → model becomes simpler → bias increases

We need to find a balance.

Maximum likelihood estimation

MLE is a method used to find the best parameter of a statistical model.
It chooses the value of the parameter that makes the observed data most likely.

What is parameter?

- A parameter is something we want to estimate.

e.g:- • Mean (μ) of a Gaussian distribution
• Probability of heads (p) in a coin
• Rate (λ) a poisson process.

What does MLE do?

MLE asks:-
"which parameter value makes the data we observed MOST likely?".

We calculate:-

- The likelihood of data for different parameter values.
- choose the parameter with highest likelihood.

- MLE attempts to find the parameter values that maximize the likelihood function.

Bayes theorem:-

$$P(\theta|\text{Data}) = \frac{P(\text{Data}|\theta)\,P(\theta)}{P(\text{Data})}$$

(likelihood)

- we need to estimate the parameters of probability distribution to test whether the given dataset follow some particular distribution.

* MLE is a method to estimate parameters of a probability distribution given observation
* The likelihood of sample x is a function of parameter θ. It can be defined as the one that parmeter θ would generate the observed data.

$$l(\theta) = p(x|\theta)$$
$$= p(x_1|\theta)\, p(x_2|\theta) \ldots p(x_n|\theta)$$

$x = \{x_1, x_2, \ldots x_n\}$

apply logirthm on b.s $\because \log(a \cdot b) = \log a + \log b$

$$\log(l(\theta)) = \log[p(x_1|\theta)\, p(x_2|\theta) \ldots p(x_n|\theta)]$$

$$L(\theta) = \log\{P(x_1|\theta) + \log p(x_2|\theta) + \ldots$$
$$+ \log[P(x_n|\theta)$$

log likelihood fn ↗

$$L(\theta) = \log f(x_1|\theta) + \log f(x_2|\theta) + \ldots$$
$$\ldots \log f(x_n|\theta)$$

Maximum likelihood estimate ↗

MLE Choose the parameters (the probability of) that maximize the chance of your data appearing.

Bayesian statistics

Bayesian statistics is a way of doing statistics where we update our belief about something when we see new data.
Its based on Baye's Theorem

Bayes Theorem :-

$$P(A|B) = \frac{P(B|A)\, P(A)}{P(B)}$$

Posterior = (Likelihood x prior) / evidence.

working :-
You start with a prior belief
you observe some data
you update your belief
Bayesian statistics = prior belief + New data → updated belief.

Example :- COVID Test positive

let :-
- 1% of population has covid
  → prior : $P(\text{Disease}) = 0.01$ $\quad P(\text{No Disease}) = 0.99$
- Test is 95% accurate
  → $P(\text{positive} \mid \text{Disease}) = 0.95$
- false positive rate = 5%.
  → $P(\text{positive} \mid \text{No disease}) = 0.05$

You test positive.
what is the chance you actually have COVID?

Apply Bayes :-

$$P(D|+) = \frac{0.95 \times 0.01}{(0.95 \times 0.01) + (0.05 \times 0.99)}$$

$$P(D|+) = \frac{0.0095}{0.0095 + 0.0495} = \frac{0.0095}{0.059} \approx 0.16$$

Interpretation:-

Even after postive test, chance of having COVID is only 16%.

→ Because the prior probability (17.1 was very low.

* It combines prior knowledge with new data to produce a posterior belief.

Supervised learning algorithms:-

It is a type of machine learning where:-

- we train the model using labeled data.
- The model learns the relationship between input (x) and output (y)
- Then the model predicts output for new/ unseen data.

example:- - predicting house price (regression)
- classing email as spam or not spam (classification)

Two main types of supervised learning

1. Regression.

TO predict a continuous numerical value.

- linear regression:- Assumes a linear relationship between input and output.
- Non-linear - used when the relationship is curved or complex.

common Algos:-

* linear regression * Polynomial regression
* Ridge & lasso Regression
* Deep Feedforward Networks.

2. classification

To predict a discrete categorical labels (a class)

- Binary classification:- predicting one of two possible classes.
- Multi-class classification:- predicting one or more than two classes

common classification Algos:-

* logistic regression * support vector Machine
* Decision Trees & Random forest
* K-Nearest Neighbours (KNN)
* Deep Feedforward Networks (DFNs)

Unsupervised learning Algorithms:-

It is a type of machine learning where:-

- No labeled data is given
- The model finds patterns, groups or structure in input data.
- It discovers hidden relationships automatically.

The model's goal is not to predict outcome but rather to model the underlying structure or distribution of data itself. It tries to answer questions like.

- How are these data points related to each other?
- Can I group similar data points together?
- Can I simply the data without loosing much information?

Major categories:-

1. Clustering 2. Dimensionality Reduction
3. Association rule.

1. Clustering :-
   To group similar data points together into clusters based on their inherent characteristics.

   * k-means * Density-Based-spatial clustering
   * Hierarchial clustering

2. Dimensionality Reduction:-
   To reduce the number of features (dimensions) in the dataset while retaining most of the important information.

   * PCA * Autoencoders, * SOM

3. Association Rules:-
   * Apriori

Used for segmentation, compression, anomaly detection and pattern discovery.

Stochastic Gradient Decent (SGD)

It is the most important optimization algo in deep learning. It's the engine that drives the traning of your deep feed-forward Networks.

The Goal: Minimizg loss:-

First lets remember the goal of training a model: we want to find the set of weights and biases (θ) that minimizes the loss function ($J(\theta)$). the loss function measures how "wrong" the model's predictions are.

Gradient descent

GD works by iteratively moving the parameters (θ) in the direction of the steepest decrease of the loss function.

- The Gradient ($\nabla J(\theta)$) is the calculated slope of the losse function, telling us the direction of steepest ascent.
- We take a step in the opposite direction of the gradient to roll dow the loss hill

The update rule is:-

$$\theta_{new} = \theta_{old} - \eta \nabla J(\theta_{old})$$

where η is the learning Rate (the size of the step).

Stochastic Gradient Descent (SGD)

The term "stochastic" means "involving a random variable" or here "randomly sampled". SGD is a variation of GD designed to much faster and more scalable for large datasets.

Problem with Basic GD

Standard Batch Gradient Descent calculates the gradient using the entire training dataset before making a single update step. If you have a million data points, you have to process all one million before moving your weights even once. This is computationally too slow.

The SGD solution:-

Stochastic Gradient Descent addresses this by estimating the gradient using only one single, randomly selected data point (or, more commonly, a small mini-batch of data) at a time.

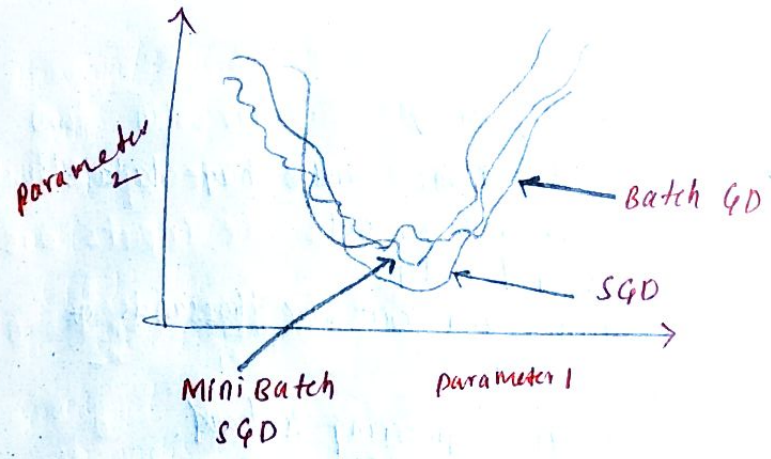
→ Batch GD :- All m training Gradient calculation
~~frequency~~ slow (one update per epoch)
very stable convergence

→ SGD :- one training example, very fast,
~~high variance, but fast~~

→ Mini-Batch SGD :- A small subset, fast,
Best Balance:-

Key Benifit of SGD / Mini Batch SGD

* speed
* Memory
* Noise/ Exploration.



Building a Machine learning algorithm:-

It involves the following steps:-

1. problem definition
2. Data collection
3. Data preprocessing
4. selecting the appropriate model
5. Choosing a loss function
6. Choosing an optimizer
7. Training the model using training data
8. Validating the model on unseen ~~test~~ validation data
9. Testing the model on unseen test data
10. Deployment of the model
11. Continuous monitoring and improvement.

Challenges Motivating Deep learning :-

Deep learning became Popular because traditional Machine learning methods were not able to handle modern, complex data & real-world problems.

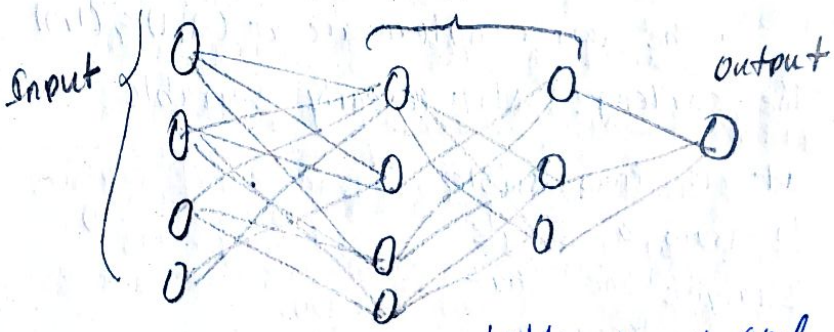
Main challenges:-

1. Too much data (Big Data)
   Traditional ML cannot handle huge datasets like images, videos, text.
2. High-dimensional data
   Images, audio and text have thousands of features - ML fails.
3. Manual feature engineering is hard.
   Old ML required manual creation of features. DL learn features automatically.
4. Unstructured data
   ML works better on tables; DL works on raw data like images, sound, and text.
5. Low accuracy for of Traditional ML
   Application like face recognition & self-driving need very high accuracy → DL performs better.
6. Complex patterns in modern tasks
   Traditional ML cannot capture deep non-linear patterns.
7. Need for end-to-end systems
   DL can do raw input → final output directly without many steps.
8. Better hardware availability
9. Growth of big datasets
10. Scalability
    DL handles millions of example easily
    ML becomes slow.

Deep ~~Forward~~ feedforward Networks (DFNs)

- Deep feedforward networks, also called feed-forward neural networks, or multilayer perceptron (MLPs), are the quintessential deep learning models.
- The goal of a feedforward network to approximate some function f^*

- A feedforward network defines a mapping $y = f(x;\theta)$ and learns the value of the parameters θ that result in the best function approximation.
- These models are called feed Forward because information flows through the function being evaluated from x, through the intermediate computations used to define f, and finally to the output y.



It has Input layer, hidden layers and output layer

Goal:-
Approximate a mapping from inputs to outputs.

Learning XOR

XOR is a basic logical function with two binary inputs (x_1, x_2) and one binary output, the output is 1 only when the inputs are different and 0 otherwise

input x_1	input x_2	output y
0	0	0
0	1	1
1	0	1
1	1	0

The failure of the single-layer Perceptron :-

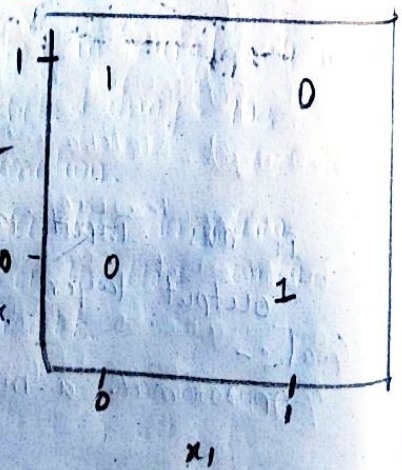
A single layer perceptron works by a single straight line (a linear decision boundary) to separate data points into two classes.

When the four XOR data points are plotted on a 2D graph, the points are arrange diagonally.

- The two '0' output are at (0,0) & (1,1)
- The two '1' output are at (0,1) & (1,0)

The challenge : NOT linearly separable

It is immpossible to draw a single straight line that correctly separates the two '1' outputs from the two '0' outputs.



Hence we can use activation function at hidden units. We apply a linear transformation on data and then apply non-linear activation function.

↓

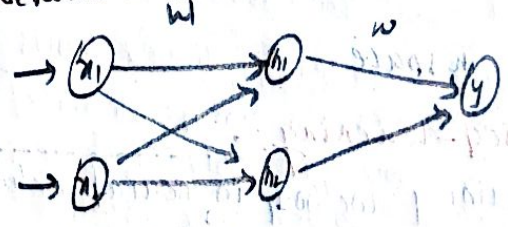
Rectified linear function

The solution :- Introducing a Hidden layer

The way to solve XOR is transform the input data into a new, higher-dimensional space where the classes become linearly separable. This is the exact function of the hidden layer in a feed forward function.

A DFN with just one hidden layer can solve the XOR problem by creating a non-linear decision boundary.

Network Diagram



By learning

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

$$X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

2. Step two multiply with W

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}$$

③ add the bias vector c

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

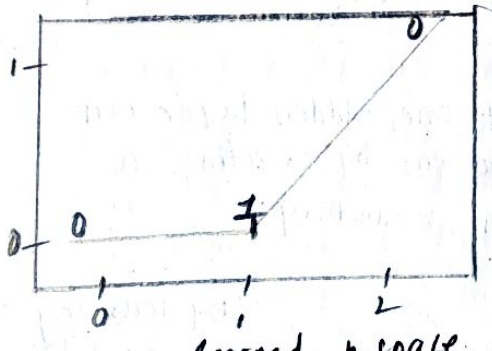
④ apply the rectified linear transformation

If −ve → 0
+ve → same

$$\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

⑤ multiplying by the weight vector w

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$



Gradient-based learning:-

It is a training method in neural networks where weights are updated using the gradient of the loss function.

The gradient indicates the direction and rate of change of the loss with respect to each weight.

→ A gradient is a loss function

so gradient tells:-

which direction to move the weights to reduce the error.

why needed?

- Neural Networks have millions of weights we need a method to adjust all these weights automatically & efficiently.

Gradient help computes:-

- How much each weight contributed to the error.
- How to change each weight to reduce error.

This is why learning uses Gradient Descent.

The core Idea:-

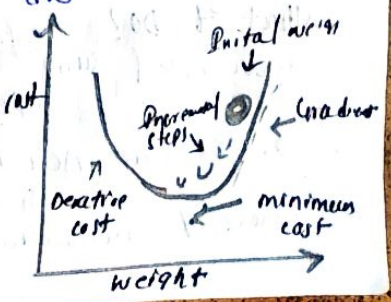
1. Forward pass
   Given Input → network → compute output
   → compare with true label → calculate loss.
2. Backward pass
   compute gradients to the loss with respect to each weight using backpropagation
3. Update weights
   use the gradient to modify weights

$$w_{new} = w_{old} - \eta \frac{dL}{dw}$$

where:-

η = learning rate

$\frac{\partial L}{\partial w}$ = gradient (slope)



Role of loss function:-

- The loss function measures how far the models prediction is from the actual target
- common loss functions
  - Mean squared Error (MSE) for regression
  - Cross-Entropy loss for classification
- The goal of gradient-based learning is to minimize this loss.

Hidden units:-

In a neural networks, Hidden units are the neurons present in the hidden layers, i.e layers between the input and output

Hidden units are neurons in the intermediate layers of a neural network. They transform input data through weighted sums and activation functions to learn complex, non-linear patterns. Hidden units enable deep networks to form meaningful interal representation and solve problems that cannot be solved by simple linear models.

What it Do?

1. Weighted sum

$$z = w_1x_1 + w_2x_2 + \dots b$$

2. Non linear Activation

$$h = \phi(z)$$

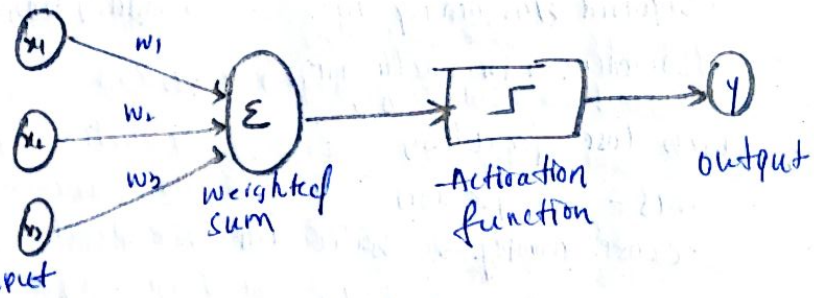
where ϕ = Activation function (ReLU, sigmoid, tanh,

Activation functions of Hidden units

- ReLU : $h = \max(0, z)$
- sigmoid
  used in logistic problems, older networks
- Tanh
  good for centered outputs (-1 to +1)

Each activation affects:-

- learning speed
- Gradient flow
- ability to represent functions.



Architecture Design

It refers to deciding the structure of a neural network, including the ~~network~~ number of layers, neurons, activation functions, connections and training strategy. Proper architecture design helps the network learn pattern efficitively, avoids overfitting / underfitting & improve performance.

1. How many layers
   - 1-hidden layer – simple problem
   - Many hidden layers – complex problems
2. How many neurons
   - more neurons → learn more patterns
   - ~~Too~~ too many → overfitting
   - Too few → underfitting
3. Which activation function
   - ReLU → commonly used
   - Sigmoid → for binary o/p
   - Softmax → for multi class o/p.
4. Which optimizer
   - SGD
   - Adam (most used)
5. which loss function
   - MSE → regression
   - cross entropy → classification
6. Regularization methods
   - To avoid overfitting
     - Dropout
     - L2 regularization.

Main layers:-

1. Input layer
2. Hidden layer
3. Output layer.

Input layer (features $x_1, x_2 \ldots x_n$)
↓
Hidden layer 1 (Neurons + ReLU)
↓
Hidden layer 2 (Neurons + ReLU)
↓
Output layer (Softmax / Sigmoid etc).

Back Propagation

Backpropagation is the algorithm used to train neural networks. First the forward pass compute the output.

Error is calculated from predicted vs actual. error is sent backward through each layer using Chain rule.

Gradient tells how much each weight caused error. Weights and biases are updated to reduce future errors.

1) forward propagation or forward pass
   - we give input to the network
   - It calculates outputs layer by layer
   - finally, it produces a prediction

e.g:-
Input → Hidden layer → output → pred = 0.8
Actual answer = 1
So the network made the error.

2) Calculate error
   error = pred − actual
3) Send error backwards
   so the error moves backward from the output layer → hidden layer → input sid.
4) Update weights.

Other differentiation Algos

A. Numerical differentiation

- Approximates the derivative using small changes in weights.

$$\frac{dL}{dw} \approx \frac{L(w+h) - L(w)}{h}$$

Pros:- simple, Good for debugging gradients

cons:- very slow
- Not suitable for large Networks

B. Symbolic Differentiation
- Used in computer algebra systems
- Derivative is computed symbolically as an expression
- Not used in DL because expressions become huge.

C. Automatic Differentiation

used in frameworks like Tensorflow, Pytorch, JAX

Two modes:-
1. Forward mode AD
2. ~~Bad~~ Reverse mode AD

Backprop is actually reverse-mode autodiff

D. BPTT - Back propagation through time

E. Real-time Recurrent learning RTRL